Explicit-Text-Classifier — Interview Deep-Dive

> **Who this is for:** me (the author), preparing to explain and defend this project in ML interviews. **How to read it:** plain English first, then technical depth. Every term is defined on first use, and there's a Glossary at the end. **Ground rules:** checked against `Explicit Content Classifier.ipynb` (cells and saved outputs), `README.md` and the git history (HEAD `ef0946c`). Anything not evidenced is marked **UNVERIFIED**.

---

0. TL;DR

- **What:** a Jupyter notebook that trains a **TF-IDF + linear SVM** text classifier on the HuggingFace dataset `valurank/Explicit_content` (news-style articles labelled `Explicit` / `Not_Explicit`). Preprocessing: lowercase, strip non-letters, spaCy lemmatise, NLTK stopwords. Then an 80/20 split and a `classification_report`.
- **The single most important fact:** the README says *"Precision (explicit class) 0.86, Recall (explicit class) 0.96"*. **That's wrong. Those are the scores of the *Not-Explicit* class.**
  - Cell 11 calls `classification_report(..., target_names=['positive', 'negative'])`. sklearn assigns names in sorted label order, so **label 0 (Not_Explicit) is printed as "positive"** and label 1 (Explicit) as "negative".
  - The supports prove it: "positive" has 151 test rows and "negative" has 64. The full dataset is 757 class-0 vs 315 class-1 (cell 7 output), so the larger group must be class 0.
  - **Actual explicit-class performance (cell 11 output): precision 0.87, recall 0.62, F1 0.73.** Overall accuracy 0.86, macro-F1 0.82.
- **Also:** the README describes **hard-negative training** and a **3-tier confidence threshold** on `decision_function` (> 0.8 / 0.3–0.8 / < 0.3). **Neither exists in the notebook.** That README was rewritten in May 2026 by a commit authored "Antigravity" (an AI coding agent), about a year after the notebook was committed. The "high recall" design story rests on the mislabelled number.
- **Course context (found in another repo):** the project report, `Report_.pdf`, was committed by mistake to the `E-Commerce-Marketplace` repo (commit `c6e7f32`). It's the **BITS F464 Machine Learning, Semester 2 2024-25, Phase-3 report**, a **team of four** (Siddhartharaja Tammana, Kaushik Voolapalli, Malisetty Vinisha, Sai Lokesh Kalva). It's based on the paper *"Machine Learning Techniques for the Detection of Inappropriate Erotic Content in Text"* (IJCIS 2020).
  - The report states "accuracy (0.86) and F1-score (0.91)" and "precision reaching a value of 0.86". The 0.91 F1 and 0.86 precision are the **Not-Explicit** class's values (the same `target_names` swap). The explicit class's F1 is **0.73**.
  - The report's own work-split table credits you with "generated the results of the paper, designed the model and the computational framework, carried out the experiments".

---

1. One-paragraph pitch (30 seconds)

"It's a classical NLP baseline for flagging explicit content in articles. I clean the text, lemmatise with spaCy, remove stopwords, build TF-IDF features over unigrams and bigrams, and train a linear-kernel SVM. On a held-out 20% split of about 1,070 de-duplicated articles it gets 86% accuracy. For the explicit class specifically, it's precise (0.87) but misses more than a third of explicit articles (recall 0.62). That's the main thing I'd fix, with class weighting, threshold tuning on the SVM margin, and ultimately a fine-tuned transformer, because bag-of-words can't distinguish *discussing* explicit content from explicit content."

(This is the honest version. Don't use the README's 0.96 recall. See §8.)

---

2. Plain-language explanation

The problem. Websites need to automatically flag articles that contain explicit or disturbing material, so a person can review or hide them. Doing that by hand doesn't scale.

The approach (a "word-counting" classifier).

1. **Clean** each article: lowercase, remove punctuation and numbers, turn words into their dictionary form ("running" → "run"; this is called *lemmatising*), and drop very common words like "the" and "is" (*stopwords*).
2. **Turn text into numbers** with **TF-IDF**. Each word or two-word phrase gets a score that is high if it's frequent *in this article* but rare *across all articles*. So distinctive words stand out.
3. **Train an SVM** (support vector machine), a classic algorithm that finds the best dividing line between the two groups in this huge word-space.
4. **Test** on 20% of articles the model never saw.

The honest result. When the model says "explicit", it's right about 87% of the time. But it only catches about 62% of the explicit articles. In content moderation, missing explicit content is usually the costlier error, so that's the number to improve.

Why it struggles. It only sees *which words appear*, not what the sentence means. "The platform removed sexually explicit posts" (a news story about moderation) and actually explicit text share words.

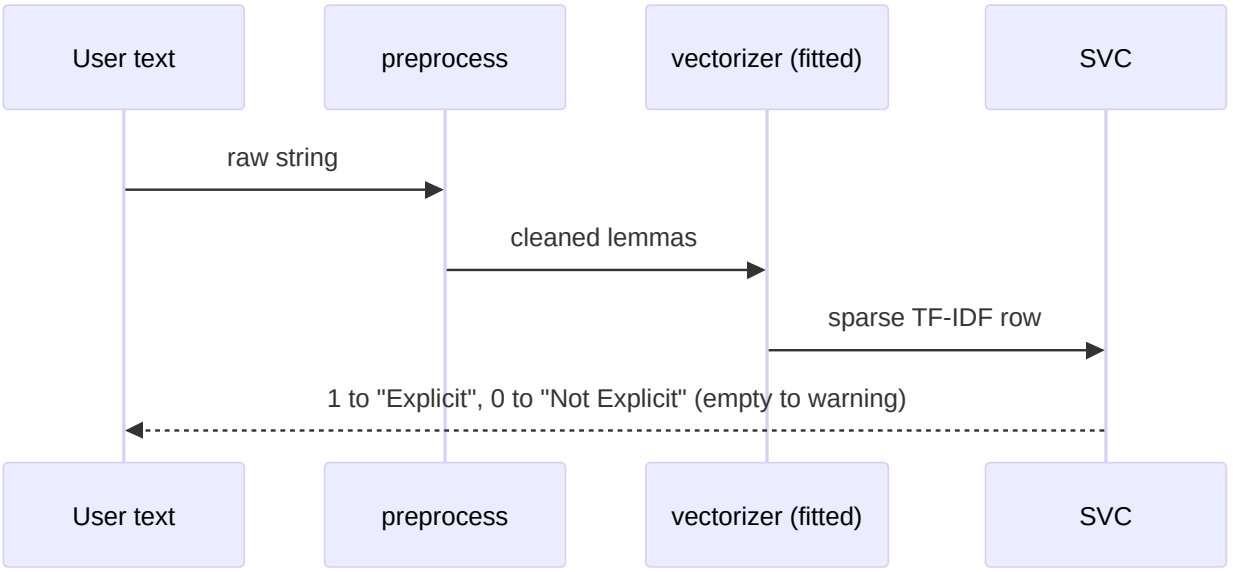
---

3. Architecture

3.1 Pipeline



3.2 Inference path



There is no server, API or saved model. Everything lives in the notebook's kernel memory.

---

4. Tech stack

Tool	Role	Why	Alternatives
`datasets` (HF)	`load_dataset("valurank/Explicit_content")`	One-line download	Manual CSV
pandas	DataFrame ops, label mapping, dedupe	Standard	polars
`re`	`[^a-z\s]` filter	Simple	spaCy tokeniser only
spaCy `en_core_web_sm`	Lemmatisation	Better lemmas than Porter stemming	NLTK WordNetLemmatizer (faster, cruder); no lemmatisation (TF-IDF bigrams often do fine without)
NLTK stopwords	Stopword list	Standard list	spaCy's stopword list; none (let IDF down-weight them)
scikit-learn `TfidfVectorizer`, `SVC`, `train_test_split`, `classification_report`	Features, model, eval	Classic, strong baseline for text	`LinearSVC` (much faster for linear kernels on sparse data), `LogisticRegression` (gives probabilities), Naive Bayes; DistilBERT/RoBERTa fine-tuning

Tool	Role	Why	Alternatives
matplotlib/seaborn	Listed in the README install line but **not used** in the notebook	—	—

---

5. Code walkthrough (cell by cell)

> Execution counts are out of order (cells ran as 94, 120, 126, **138**, 128, **139**, …). The saved outputs reflect a kernel state built from non-linear execution. E.g. cell 3's output already shows a `clean_text` column, which cell 5 creates later.

Cell	What it does	Notes
0	Imports; `nltk.download('stopwords')`; `spacy.load("en_core_web_sm")`	Output shows a Windows path `C:\Users\Akhil\…`, so it was run on a machine with user "Akhil"
1–2	`load_dataset` → `ds['train'].to_pandas()`	Only a `train` split is used
3	Map `Explicit` → 1, `Not_Explicit` → 0, cast to int	Output shows 1,189 index entries
4	**`preprocess(text)`**: lowercase → `re.sub(r'[^a-z\s]', '', text)` → spaCy → keep `token.lemma_` if `token.text` not in stopwords/punctuation	Regex runs **before** spaCy: contractions break ("didn't" → "didnt"), numbers vanish, sentence boundaries go. Stopword check uses the *surface* form, then emits the *lemma*. Negations like "not" are NLTK stopwords, so they're removed
5	`drop_duplicates(subset=['Article'])`, then apply `preprocess`	Prints `df.size` = 3216 both before and after (rows × 3 columns = 1,072 rows). Dedupe had already happened in an earlier run
7	`value_counts`: **0 → 757, 1 → 315**	~29% positive, so imbalanced
8	`TfidfVectorizer(ngram_range=(1,2))` **fit on the full dataset**	Vocabulary and IDF use test documents: mild transductive leakage
10	`train_test_split(X, y, test_size=0.2, random_state=42)` → `SVC(kernel='linear', C=100)`	**Not stratified**. **C=100** means very weak regularisation on ~860 training docs with a huge bigram vocabulary. No CV or tuning
11	`classification_report(y_test, y_pred, target_names=['positive','negative'])`, plus first `detect_explicit`	**Names swapped** (see §0). "She trembled as he gently ran his fingers…" → Not Explicit
12	Second `detect_explicit`: empty-input guard, binary label	Returns a **string only**. The README's `label, score = detect_explicit(text)` would fail
14	Re-splits *raw* texts with the same `random_state` to print each test article with true vs predicted label	Works only because the row count and seed match. Fragile. Prints the whole test set

---

6. Core concepts & theory

6.1 TF-IDF

- **Term frequency** $tf(t, d)$ is the count of term *t* in document *d*.
- **Inverse document frequency.** sklearn's default (`smooth_idf=True`) is $idf(t) = \ln \frac{1+n}{1+df(t)} + 1$.
- The weight is $w_{t,d} = tf \cdot idf$, and then each document vector is L2-normalised (`norm='l2'`).
- **Bigrams** capture short phrases ("sexual assault" ≠ "assault"), but they blow up the vocabulary. With `min_df=1` that's likely 100k+ features for about 1k documents, i.e. $p \gg n$.

6.2 Linear SVM

- It finds w, b that maximise the margin $2/\|w\|$ while penalising violations:

  $$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 + C \sum_i \max(0,\ 1 - y_i(w^\top x_i + b))$$

  The second term is the **hinge loss**.
- **C** trades margin width against training errors. Large C (100) means "fit the training set almost perfectly". With $p \gg n$, the data is almost certainly linearly separable, so C hardly matters past a point. The model is effectively the hard-margin SVM, which risks overfitting.
- `decision_function` returns $w^\top x + b$, a **signed distance to the hyperplane**, not a probability. The README's thresholds of 0.3 and 0.8 on this score are arbitrary unless calibrated (Platt scaling via `CalibratedClassifierCV`, or `SVC(probability=True)`).
- `SVC(kernel='linear')` uses libsvm (a dual solver, roughly O(n²–n³)). `LinearSVC` (liblinear, primal) is the standard faster choice for sparse text.

6.3 Class imbalance & thresholds

- With 29% positives, a default threshold at 0 is not recall-optimal. Options:
  - `class_weight='balanced'` scales C per class by $n/(k \cdot n_c)$;
  - shift the threshold on `decision_function` using a precision-recall curve from validation;
  - resample.

6.4 Metrics

- **Precision** $= TP/(TP + FP)$, **Recall** $= TP/(TP + FN)$, $F_1 = 2PR/(P + R)$.
- F_β with β = 2 weights recall more, which suits moderation.
- **Reading a classification report:** rows are classes in sorted label order. `target_names` only *renames* them, so a wrong order silently swaps the labels.
- **PR-AUC** (average precision) is more informative than ROC-AUC on imbalanced data.

6.5 Why bag-of-words fails on meta-discussion

- TF-IDF is order-free and context-free. "Explicit content was removed" contains the same tokens as content that *is* explicit.
- Contextual encoders (BERT-family) represent each token conditioned on the whole sentence, so they can learn the difference between *mention* and *use*.

7. Data

- **Source:** HF `valurank/Explicit_content`, `train` split.
  - 1,189 rows (cell 3 index), with columns `Article` and `Category`.
  - **1,072 after de-duplication**: 757 Not_Explicit, 315 Explicit.
- **What "explicit" means here is ambiguous.** The first "Explicit" rows shown are news stories (a sentencing in Wisconsin, a southern-California rescue, political commentary). The label seems to mean *disturbing or sensitive news content* (crime, violence) rather than sexual content. The ad-hoc test sentences in cells 11–12 test *sexual* content, which is a different concept. **Clarify this before an interview.**
- **Preprocessing:** see cell 4. No length normalisation, no handling of very long articles.
- **Split:** a single random 80/20 split (215 test rows: 151 class 0, 64 class 1). Not stratified, no validation set, no CV.
- **Leakage:**
  1. The TF-IDF fit includes test docs (mild).
  2. Near-duplicates: only exact `Article` duplicates were removed. Syndicated news often appears in slightly edited form, which can inflate test scores.
  3. Hidden-state notebook execution: results depend on cell order.

---

8. Evaluation & results

From the saved output of cell 11 (labels corrected):

Class	Precision	Recall	F1	Support
Not Explicit (0), printed as "positive"	0.86	**0.96**	0.91	151
Explicit (1), printed as "negative"	**0.87**	**0.62**	**0.73**	64
Accuracy			0.86	215
Macro avg	0.86	0.79	0.82	215

- **Rough confusion matrix** (back-computed from these rounded numbers, so approximate): explicit recall 0.62 × 64 ≈ 40 TP and ≈ 24 FN; precision 0.87 means ≈ 6 FP. So about **24 explicit articles were missed** for every 6 false alarms. That's the opposite of the "high recall" story.
- **Baseline:** predicting "Not Explicit" for everything scores 151/215 = **70.2% accuracy**. The model's 86% is a real but modest lift. Accuracy is the wrong headline metric here.
- **Soundness:** a single unstratified split of 215 docs (64 positives), no CI. The 95% CI on recall 0.62 with n = 64 is roughly ±0.12. No CV, the vectoriser is fit on everything, and there's no hyper-parameter search.
- **Qualitative checks (cells 11–12):** 3 anecdotal sentences. "She trembled... candlelight" → Not Explicit, "...having sex..." → Explicit. These are single sentences, out of distribution relative to the news-article training data.

---

9. Design decisions & trade-offs

Decision	Gained	Lost
TF-IDF + linear SVM	Fast, interpretable (inspect top-weighted n-grams), strong small-data baseline	No context; can't separate mention vs use
spaCy lemmatisation	Merges inflections	Slow (full pipeline incl. parser/NER for every article; could disable components); applied after destructive regex
Bigrams, `min_df=1`	Captures phrases	Huge sparse vocabulary; overfitting risk with C=100
`SVC(kernel='linear')` instead of `LinearSVC`	Same model family	Much slower; no probabilities
Single notebook	Easy to share	Out-of-order execution; no saved artefacts; not reusable
No class weighting	Simplicity	Explicit recall only 0.62

10. Honest weaknesses

1. **Mislabelled headline metric.** The README reports the Not-Explicit class's P/R as the Explicit class's. The true explicit recall is 0.62, not 0.96.
2. **README describes features that don't exist** (hard negatives, 3-tier `decision_function` thresholds, `detect_explicit` returning `(label, score)`, file name `Explicit_Content_Classifier.ipynb` with underscores).
3. **The label concept is unclear and mismatched** with the qualitative tests (sexual-content sentences vs a news dataset).
4. **No stratification, CV, tuning, or class weighting.** C=100 is unjustified.
5. **TF-IDF fit before the split** (mild leakage).
6. **Out-of-order notebook execution** and hidden state. There are no saved models, so it can't be deployed.
7. **Preprocessing order** (regex before spaCy) degrades lemmatisation and kills negations and contractions.
8. **Tiny test set** (64 positives), so every metric has wide uncertainty.
9. **Provenance:** the notebook ran on a machine with user `Akhil` (cell 0 output), the README was rewritten by an AI agent, and the course report shows it was a **4-person team project** (BITS F464). Be clear about your own contribution: the report credits you with generating results, designing the model and running experiments.

11. How I'd improve it (prioritised)

#	Change	Effort	Impact
1	**Fix the report:** `target_names=['Not Explicit','Explicit']` and correct the README numbers	5 min	Honesty; avoids an interview disaster
2	sklearn `Pipeline(TfidfVectorizer, LinearSVC/LogReg)` + **stratified 5-fold CV** + grid over C, `min_df`, n-grams, `class_weight`	0.5 day	Leak-free, tuned, with error bars
3	**Threshold tuning for recall:** calibrate (`CalibratedClassifierCV`), pick the threshold at a target recall (e.g. 0.9) from validation PR curves, *then* implement the 3-tier review band the README describes	0.5 day	Matches the moderation objective
4	Report PR-AUC, F2, confusion matrix; majority-class baseline; bootstrap CIs	2 hours	Proper evaluation
5	Near-duplicate detection (MinHash / TF-IDF cosine > 0.9) before splitting	2 hours	Prevents inflated scores
6	Fix preprocessing: run spaCy on raw text with `disable=["parser","ner"]`, keep negations, or skip lemmatisation and compare	2 hours	Cleaner features, 5–10× faster
7	Hard-negative set (moderation policies, legal text) as a separate *challenge* test set	1 day	Measures the known failure mode
8	Fine-tune DistilBERT/RoBERTa with class weights; compare on the same CV folds	1–2 days	Likely large recall gain on context-dependent cases
9	Export a script/package with a saved pipeline (`joblib`) and a small FastAPI endpoint	0.5 day	Usable artefact

---

12. How to run it

```
python -m venv venv && source venv/bin/activate
pip install pandas scikit-learn spacy nltk datasets jupyter
python -m spacy download en_core_web_sm
python -c "import nltk; nltk.download('stopwords')"
jupyter notebook "Explicit Content Classifier.ipynb"   # note: spaces in the real file name
```

Run cells **top to bottom** (the saved outputs came from out-of-order execution). Needs internet access to the HuggingFace Hub. **Env vars:** none (optionally `HF_HOME` for the cache).

---

13. Resume bullets (truthful)

- Built a **TF-IDF (uni+bigram) + linear-SVM** text classifier for explicit-content detection on ~1.1k news articles, with a spaCy/NLTK preprocessing pipeline; **86% accuracy, explicit-class precision 0.87 / recall 0.62** on a held-out 20% split.
- Diagnosed a **label-ordering error in the evaluation report** that had inverted the reported per-class metrics, and redesigned evaluation around **recall-oriented thresholding, stratified CV and PR-AUC** for moderation use. *(Claim the second half only once it's implemented.)*

> ⚠️ Remove "recall 0.96" from anything public.

Glossary

- **NLP**: natural language processing.
- **Lemmatisation**: reducing a word to its dictionary form (better → good, running → run).
- **Stopwords**: very common words usually removed (the, is, and).
- **TF-IDF**: term frequency × inverse document frequency; a weighting that highlights distinctive words.
- **n-gram**: a sequence of n tokens (bigram = 2 words).
- **SVM**: support vector machine; finds a maximum-margin separating hyperplane.
- **C (SVM)**: regularisation trade-off; large C = less regularisation.
- **Decision function**: a signed distance to the SVM boundary (not a probability).
- **Precision / recall / F1**: of predicted positives, how many are right / of real positives, how many are found / their harmonic mean.
- **Stratified split**: a split that preserves class proportions.
- **PR-AUC**: area under the precision-recall curve.
- **Calibration**: making scores behave like true probabilities.
- **Hard negatives**: negative examples deliberately chosen to look like positives.

Explicit-Text-Classifier — Interview Questions & Model Answers

> Companion to `Explicit-Text-Classifier.md`. First-person, 30–90 s answers. 🟢 Easy · 🟡 Medium · 🔴 Hard

If you only prep 10: A1, A3, C3, D1, D3, E1, E2, G1, H1, and all hostile follow-ups (X1 especially).

(a) Project overview & motivation

A1. 🟢 Describe the project.

> It's a classical NLP baseline for explicit-content detection. I load about 1,200 labelled articles from HuggingFace, de-duplicate to 1,072, clean and lemmatise them, build TF-IDF features over unigrams and bigrams, and train a linear SVM. On a 20% held-out split it gets 86% accuracy. On the explicit class it has 0.87 precision but only 0.62 recall, and improving that recall is the main next step.

A2. 🟢 Why this problem?

> Content moderation is a high-volume, high-stakes classification problem with asymmetric costs. Missing explicit content is usually worse than a false alarm. It's a good setting for thinking about thresholds and metrics beyond accuracy.

A3. 🟡 What are your results?

> Accuracy 0.86, macro-F1 0.82. For the explicit class: precision 0.87, recall 0.62, F1 0.73. For non-explicit: precision 0.86, recall 0.96. I want to flag that an earlier README version reported 0.86/0.96 as the explicit class's numbers. That came from a label-ordering mistake in `classification_report`, which I've since diagnosed.

A4. 🟡 Why start with TF-IDF + SVM instead of a transformer?

> It's the right baseline: fast, interpretable (you can read the top-weighted n-grams), and strong on small text datasets. It tells you how much a transformer actually buys you. With about a thousand documents, a linear model is also less likely to overfit than a fine-tuned 66M-parameter model, though DistilBERT is the obvious next comparison.

A5. 🟢 What would you do differently?

> Start with the evaluation design: a stratified CV pipeline, a recall target, and a labelled challenge set of hard negatives. Then build models to that spec.

(b) Architecture & design

B1. 🟢 Walk through the pipeline.

> Load the HF dataset to pandas, map labels to 1/0, drop duplicate articles, preprocess (lowercase, regex to letters only, spaCy lemmas, NLTK stopword removal), TF-IDF with (1,2)-grams, an 80/20 split, `SVC(kernel='linear', C=100)`, a classification report, and a `detect_explicit` function for single strings.

B2. 🟡 Why lemmatise? Is it necessary with TF-IDF?

> It merges inflections (killed/killing → kill), which reduces sparsity on a small dataset. But it's not always beneficial. Bigrams plus IDF often work just as well without it, and spaCy lemmatisation is the slowest step. I'd A/B it in CV, and if I keep it, disable the parser and NER components.

B3. 🟡 Why bigrams?

> Phrases carry meaning single words don't, like "sexual assault" or "graphic violence". The cost is a much bigger vocabulary. With `min_df=1` and about a thousand documents, most bigrams appear once, which adds noise. I'd tune `min_df` and `max_features`.

B4. 🟡 How would you design the "needs review" tier?

> Calibrate the SVM scores (Platt scaling via `CalibratedClassifierCV`), then pick two thresholds on validation data. The upper one gives high precision for auto-flagging. The lower one is set to hit a recall target like 95%, and everything between goes to human review. You then report the volume going to review as a cost. The README describes this, but the notebook doesn't implement it yet.

B5. 🔴 How would you productionise it?

> Wrap preprocessing, vectoriser and model in one sklearn `Pipeline`, serialise it with joblib, and serve it behind a FastAPI endpoint returning label, calibrated score and tier. Log inputs and decisions for auditing. Monitor drift through the score distribution and review rates. Retrain periodically on reviewed cases, which is effectively active learning.

(c) Code-level / implementation

C1. 🟡 Walk me through `preprocess`.

> Lowercase, then regex `[^a-z\s]` removes everything except letters and whitespace. Then spaCy parses it, and I keep `token.lemma_` for tokens whose text isn't a stopword or punctuation. Two issues. The regex runs before spaCy, so contractions break ("didn't" → "didnt") and spaCy's lemmatiser sees unnatural text. And NLTK's stopword list includes "not" and "no", so negation is lost.

C2. 🟢 Why `drop_duplicates` before splitting?

> Otherwise the same article can land in both train and test, which inflates scores. It removed 117 exact duplicates (1,189 → 1,072). Near-duplicates, like syndicated news with small edits, aren't caught.

C3. 🔴 Explain the `target_names` bug.

> `classification_report` orders rows by sorted label: 0, then 1. `target_names` just renames them in that order. I passed `['positive','negative']`, so class 0 (Not Explicit) was printed as "positive" and class 1 (Explicit) as "negative". The supports give it away: 151 vs 64, while the data is 757 class-0 to 315 class-1. So the 0.96 recall belongs to the non-explicit class. The real explicit recall is 0.62.

C4. 🟡 Is fitting TF-IDF on all data before splitting a problem?

> It's mild leakage. The vocabulary and IDF weights include test documents, so test terms aren't "unseen", and IDF reflects the test distribution. The effect is usually small, but it's free to avoid: put the vectoriser in a Pipeline and fit inside each CV fold.

C5. 🟡 `SVC(kernel='linear')` vs `LinearSVC`?

> Same model family, different solvers. `SVC` uses libsvm's dual solver, which scales roughly quadratically to cubically with the number of samples. `LinearSVC` uses liblinear in the primal, which is much faster on large sparse text. `SVC` also doesn't give probabilities unless `probability=True`, and that runs an internal CV for Platt scaling.

C6. 🟢 What does `detect_explicit` return?

> In the final version, a string: "Explicit", "Not Explicit", or an empty-input warning. It doesn't return a score, so the README's `label, score = detect_explicit(text)` example would fail.

C7. 🟡 The notebook execution counts are out of order. Why does that matter?

> Saved outputs might depend on state from cells run in a different order, or on cells since edited. For example, cell 3's output shows `clean_text`, which cell 5 creates. For reproducibility you should restart and run all, or move the logic into a script.

(d) Underlying ML/CS theory

D1. 🟡 Explain TF-IDF.

> Term frequency counts how often a term appears in a document. Inverse document frequency down-weights terms that appear in many documents. sklearn uses ln((1+n)/(1+df)) + 1. Multiply them and L2-normalise each document vector. Distinctive terms get high weight, and ubiquitous ones like "said" get low weight.

D2. 🟡 Explain the SVM objective and what C does.

> Minimise ½‖w‖² + C·Σ hinge loss, where hinge = max(0, 1 − y·f(x)). The first term maximises the margin, and the second penalises points on the wrong side or inside the margin. Large C means a smaller margin and fitting training points more tightly, i.e. less regularisation. C = 100 on about 860 docs with far more features than docs is close to a hard-margin SVM, which risks overfitting.

D3. 🔴 Your data has p ≫ n. What does that imply?

> With more features than samples, the training data is almost always linearly separable, so a linear SVM can reach zero training error for many C values. Generalisation then depends on the margin and on regularisation, so C, `min_df` and feature selection need tuning by CV. Simpler regularised models like logistic regression with L2 often generalise just as well.

D4. 🟡 Why isn't `decision_function` a probability?

> It's the signed distance w·x + b to the hyperplane. It ranks examples, but its scale is arbitrary and depends on C and the feature scaling. To threshold it meaningfully you either tune thresholds on validation data directly or calibrate it into probabilities (Platt scaling fits a sigmoid; isotonic regression fits a monotone step function).

D5. 🟡 Why can't bag-of-words tell "discussing" from "being" explicit content?

> It discards word order and context. The vector for "the platform banned explicit sexual content" overlaps heavily with explicit content itself. Contextual models like BERT encode each token

> conditioned on the full sentence, so they can learn mention-vs-use.

D6. 🟢 Precision vs recall in moderation?

> Recall: of all explicit items, what fraction we catch. Precision: of everything we flag, what fraction is actually explicit. Moderation usually prioritises recall, sometimes with a human-review tier to protect precision. F2 or recall-at-fixed-precision are good single numbers.

(e) Evaluation & metrics

E1. 🔴 Is your evaluation sound?

> Partly. Exact duplicates are removed before splitting, which is good. But it's a single unstratified split with only 64 explicit test examples, so recall has a CI of about ±0.12. TF-IDF is fit on everything, there's no hyperparameter search or CV, accuracy is the headline even though a majority-class baseline gets 70%, and the report mislabelled the classes. I'd move to stratified 5-fold CV inside a Pipeline and report PR-AUC, F2 and recall at a fixed precision.

E2. 🟡 What's the majority baseline and why does it matter?

> Always predicting "Not Explicit" gives 151/215 ≈ 70% accuracy. So 86% is a real but modest gain. And accuracy hides that a third of explicit articles are missed.

E3. 🟡 How would you choose an operating threshold?

> On validation folds, compute the precision-recall curve from calibrated scores, pick the threshold that achieves the target recall (say 0.9), report the precision at that point, and then check it on the held-out test set. Ideally the target comes from business costs, i.e. the cost of a miss vs the cost of a review.

E4. 🟡 What test sets would you add?

> A hard-negative challenge set (policy documents, legal text, news about moderation) to measure false positives on meta-discussion. A set of short user-generated texts, because the training data is long news articles. And per-topic slices to catch spurious correlations, like crime news being labelled explicit.

E5. 🟢 How do you get confidence intervals?

> Bootstrap the test predictions (resample with replacement 1,000 times and take percentiles), or use Wilson intervals for proportions like recall. CV across folds also gives a spread.

(f) Scaling, production & system design

F1. 🟡 How would this scale to millions of documents per day?

> TF-IDF + linear model inference is extremely cheap: a sparse dot product. The bottleneck is spaCy. I'd drop lemmatisation or use `nlp.pipe` with disabled components and multiprocessing. HashingVectorizer avoids storing a vocabulary and makes the transform stateless.

F2. 🟡 How would you handle drift?

> Monitor the score distribution and the flag rate, sample flagged and unflagged items for human audit, and retrain on newly reviewed data. Vocabulary drift (new slang) is a big problem for TF-IDF, which is another argument for subword models.

F3. 🔴 Design a two-stage moderation system.

> Stage 1: a cheap, high-recall model (this one, or a small transformer) on everything. Stage 2: a heavier model (a fine-tuned RoBERTa or an LLM judge) on stage-1 positives and borderline cases. Stage 3: human review for the uncertain band. Measure end-to-end recall, review volume and latency. Close the loop by feeding reviewed items back as training data.

F4. 🟢 What artefacts are needed at inference?

> The fitted vectoriser (vocabulary + IDF), the model, the exact preprocessing code and versions (spaCy model, NLTK stopword list), and the threshold(s). Bundle them in one serialised Pipeline to avoid training/serving skew.

(g) Failure modes & debugging

G1. 🔴 Your report says recall 0.96 for "positive" but your product misses lots of explicit articles. How do you debug?

> Check the confusion matrix with explicit label ids, not names. Then check support counts against class frequencies: 151 vs 64 when the data is 70% class 0 means "positive" is class 0. Then

> `labels=[0,1]` with the correct `target_names`. Root cause: renamed rows. Lesson: always print `confusion_matrix(y_test, y_pred, labels=[0,1])` alongside the report.

G2. 🟡 A moderation policy page gets flagged as explicit. Why?

> It shares vocabulary with explicit text, and TF-IDF has no notion of context. Mitigations: hard-negative training examples, a contextual model, or a review tier for borderline scores.

G3. 🟡 "She trembled as he ran his fingers..." is predicted Not Explicit. Why?

> No strong explicit tokens survive, and the training data is long news articles where "explicit" seems to mean crime and violence coverage. A single literary sentence is out of distribution. It's also unclear the dataset's label even covers this kind of text.

G4. 🟢 Re-running the notebook gives different numbers. Why?

> Out-of-order execution and hidden state. `train_test_split` is seeded, but if dedupe or preprocessing ran differently, the rows change. Restart and run all, or convert to a script.

G5. 🟡 Preprocessing is slow. Why?

> spaCy's full pipeline (tagger, parser, NER) runs per article via `apply`. Use `nlp.pipe(texts, batch_size=…, n_process=…)` with `disable=["parser","ner"]`.

(h) "Why didn't you..." / alternatives

H1. 🟡 Why not use class weights?

> I should have. `class_weight='balanced'` would trade some precision for higher explicit recall, which is what moderation needs. Combined with threshold tuning, it's the cheapest fix for the 0.62 recall.

H2. 🟡 Why not logistic regression?

> It's a great alternative: similar accuracy on text, natively probabilistic outputs for thresholding, and fast. I'd include it in the same CV comparison.

H3. 🟡 Why not fine-tune a transformer?

> It's the planned next step. The dataset is small, so I'd use DistilBERT or RoBERTa-base with class weights, a few epochs, and early stopping on CV folds, then compare on identical splits.

H4. 🟢 Why remove stopwords?

> It's habit, and it shrinks the vocabulary. But IDF already down-weights common words, and removing "not"/"no" hurts. I'd test with and without.

H5. 🟡 Why `C=100`?

> Honestly, it wasn't tuned. It should be chosen by CV, likely somewhere in 0.1–10 for normalised TF-IDF features.

(i) Behavioural (STAR)

I1. 🔴 Tell me about a mistake in your reported results.

> **S:** My README reported 0.96 recall on the explicit class. **T:** While preparing to defend the project, I re-checked the evaluation. **A:** The support counts didn't match the class balance, so I traced it to `target_names` renaming sorted labels in the wrong order. The 0.96 was the non-explicit class. **R:** I corrected the numbers (explicit recall 0.62) and rewrote the narrative around improving recall. Now I always print a confusion matrix with explicit label ids. I'd much rather catch that myself than have someone else catch it.

I2. 🟡 Tell me about choosing a baseline.

> **S:** A small labelled text dataset. **T:** Get a credible first model quickly. **A:** I went with TF-IDF + linear SVM: fast to train, easy to inspect, and known to be strong on small text tasks. **R:** 86% accuracy in minutes, which gave a clear bar for any transformer to beat, and made the recall gap obvious.

I3. 🟡 Tell me about working with an ambiguous label definition.

> The dataset's "Explicit" class turned out to include crime and violence news, while my mental model was sexual content, which is what my test sentences checked. I'd now start by writing a labelling guideline, reading 50 random examples per class, and agreeing on the definition before modelling.

I4. 🟢 Tell me about collaborating on this.

> It was a four-person team project for BITS F464 Machine Learning, reproducing a paper on detecting inappropriate content in text. Per our report's work split, I generated the paper's results, designed the model and framework, and ran the experiments. Teammates proposed extensions (BERT, LIME/SHAP, adversarial robustness), verified results, and wrote up the report. The notebook outputs also show it ran on a teammate's machine. *(Adjust to what you actually did.)*

Hostile follow-ups

X1. 🔴 "Your README claims 0.96 recall on explicit content. Your notebook shows 0.62. Which is it?"

> 0.62. The 0.96 is the non-explicit class. It was mislabelled because `target_names` renames rows in sorted label order, and I passed them in the wrong order. The supports, 151 and 64, prove which is which. I've corrected it.

X2. 🔴 "The README says you used hard negatives and a 3-tier threshold. Where are they?"

> They're not in the notebook. That README section describes the intended design, not the implementation. It shouldn't have been written in the past tense. Implementing the threshold tiers properly needs calibration and a validation set, which I'd add first.

X3. 🔴 "C=100, no CV, single split, 64 positives. Isn't this just a lucky split?"

> It might be. The recall CI is about ±0.12. I'd rerun with stratified 5-fold CV inside a Pipeline, tune C and class weights, and report the mean ± std. I expect explicit recall to move meaningfully with class weighting alone.

X4. 🔴 "Your 'explicit' class is mostly crime news. Does your model detect explicit content at all?"

> It detects whatever the dataset's annotators meant by "Explicit", which looks like sensitive or violent news. My ad-hoc sexual-content sentences test a different concept, so they're not a fair check. Defining the label precisely is step zero for a real moderation system.

X5. 🟡 "Why is your stopword list removing 'not'?"

> NLTK's English stopword list includes negations. For sentiment or moderation that's harmful: "not explicit" becomes "explicit". I'd use a custom list or skip stopword removal.

X6. 🟡 "Who is Akhil? And your course report says F1 0.91. Which is right?"

> *(Answer truthfully about Akhil.)* On the metric: the course report repeated the same label mix-up. 0.91 is the non-explicit class's F1, and the explicit class's F1 is 0.73. The report is in the wrong repo (E-Commerce-Marketplace). I'd move it here with a corrected results table.

X7. 🔴 "Why should we believe you understand ML evaluation if your own project mislabelled its main metric?"

> Because I found it, understood exactly why it happened, and changed my process: explicit label ids, a confusion matrix next to every report, and sanity checks of supports against class frequencies. Mistakes like this are common in practice. What matters is having checks that catch them.

X8. 🟡 "What's the first thing you'd do with one more day?"

> Pipeline + stratified CV + class weights + calibrated thresholds tuned for 90% recall. Then a hard-negative challenge set to measure false positives on meta-discussion.